

RefactorTactics — Product Requirements Document

Executive summary

RefactorTactics è un gioco tattico competitivo a turni simultanei, progettato per PC con Unreal Engine 5.8, in cui due squadre pianificano le proprie azioni nello stesso intervallo temporale e ne osservano poi la risoluzione sul campo. Il riferimento iniziale è la leggibilità tattica di *Atlas Reactor*, ma il prodotto se ne distingue attraverso tre pilastri: una mappa multilivello che partecipa attivamente al combattimento, una personalizzazione orizzontale profonda degli eroi e una pianificazione collaborativa visibile in tempo reale soltanto ai compagni.

La mappa non è uno sfondo. È un grafo tattico composto da celle dotate di altezza, terreno, copertura, visibilità, stato ambientale, collegamenti verticali ed elementi interattivi. Abilità e azioni possono incendiare, congelare, allagare, elettrificare, oscurare, distruggere o collegare zone, modificando durante la partita costi di movimento, linee di tiro e percorsi disponibili.

Il movimento non usa come autorità tattica una normale ricerca del percorso più corto. Un pathfinder semantico consulta una serie di **cost provider** che valutano una cella in relazione all'eroe, alla build, agli effetti attivi, al pericolo e agli obiettivi. Unreal Navigation può eventualmente assistere il movimento visivo e la collisione, ma il grafo tattico di RefactorTactics resta la fonte autorevole delle decisioni.

Gli eroi mantengono un'identità leggibile, ma possono essere configurati mediante moduli di abilità, talenti, specializzazioni, equipaggiamento tattico, affinità con la mappa e perk di squadra. La progressione deve ampliare le opzioni senza produrre vantaggi numerici permanenti nel PvP.

Durante la pianificazione, ogni giocatore vede percorso, destinazione, bersaglio, area d'effetto, stato di conferma ed etichetta d'intenzione dei compagni. Questi dati non devono semplicemente essere nascosti dall'interfaccia nemica: **non devono essere inviati ai client avversari**.

Il supporto alle mod è deliberatamente l'ultima macro-feature. Sarà costruito su un'architettura data-driven fin dall'inizio, ma esposto agli utenti solo dopo la stabilizzazione di regole, serializzazione, networking, versionamento e sicurezza. Unreal offre sistemi utili come plugin modulari, Game Features, Asset Manager, Primary Data Assets e Data Registries; questi sono building block, non un sistema completo e sicuro di distribuzione delle mod, che dovrà quindi essere progettato specificamente per RefactorTactics. ¹

Decisione tecnica primaria: il progetto principale userà **Blueprint per iterazione, UI e contenuti**, con un **piccolo core C++** per grafo, pathfinding, risoluzione del turno, replica selettiva e caricamento dei contenuti. La documentazione Epic descrive Blueprint e C++ come strumenti complementari e raccomanda architetture data-driven; Lyra è un riferimento ufficiale utile per modularità, multiplayer e Gameplay Ability System. ²

C# non sarà una dipendenza del runtime MVP. UnrealSharp e UnrealCLR sono integrazioni open source di terze parti che permettono di usare .NET, ma introducono dipendenza da middleware,

compatibilità da verificare a ogni versione dell'engine e rischi aggiuntivi per packaging e piattaforme. UnrealSharp dichiara supporto a .NET 10 e hot reload, mentre UnrealCLR integra .NET 6; nessuno dei due rappresenta il percorso gameplay principale documentato da Epic. C# rimane comunque molto utile per prototipi algoritmici, generatori di dati, validatori, tooling e test esterni. ³

Aspetto	Baseline ispirata ad Atlas Reactor	RefactorTactics
Turni	Pianificazione e risoluzione simultanee	Pianificazione simultanea, risoluzione autoritativa a eventi e ambiente attivo
Mappa	Arena prevalentemente statica	Grafo multilivello, celle semantiche, superfici, collegamenti e trasformazioni
Movimento	Percorso principalmente geometrico	Percorso valutato per eroe, build, rischio, terreno e stato del grafo
Personaggi	Eroi fortemente definiti, personalizzazione contenuta	Identità forte più moduli, talenti, gadget, affinità e specializzazioni
Coordinazione	Ping e comunicazione standard	Intenti, traiettorie, AoE, stato, avvisi, disegno e ghost condivisi
Informazioni private	Azioni nemiche nascoste	Piani alleati replicati solo alla squadra, mai inviati agli avversari
Mappa e abilità	Interazioni limitate	Le abilità possono aggiungere, rimuovere o modificare nodi, archi e superfici
Modding	Non centrale	Data-only mod, mappe e contenuti estensibili come ultima feature

Visione di prodotto e pubblico

Visione: creare un gioco tattico in cui il piano della squadra, la lettura tridimensionale della mappa e la configurazione degli eroi contino più della velocità meccanica o del tempo investito nello sblocco di statistiche.

Promessa al giocatore: “Costruisci il tuo eroe, modifica il campo di battaglia e orchestra un piano che tutti i tuoi compagni possono capire prima che il turno esploda.”

Obiettivi di prodotto

Obiettivo	Interpretazione operativa	Indicatore proposto
Profondità leggibile	Molte interazioni, ma con anteprime comprensibili	Almeno l'80% dei playtester sa spiegare perché un'azione ha avuto un certo risultato
Mappa protagonista	Terreno e verticalità cambiano realmente le decisioni	Almeno il 30% delle azioni in un match interagisce con una cella o con un elemento di mappa

Obiettivo	Interpretazione operativa	Indicatore proposto
Coordinazione senza voce	Una squadra casuale può costruire un piano comprensibile	Almeno il 70% dei turni contiene due o più intenti alleati compatibili
Build orizzontali	Più stili validi senza incremento permanente di potenza	Nessun modulo competitivo richiede progressione numerica permanente
Partite dense	Ridurre attese e turni "vuoti"	Match target di circa 20-30 minuti
Apprendimento progressivo	Il progetto stesso funge da percorso didattico UE	Ogni milestone produce una build giocabile e una competenza tecnica verificabile
Moddabilità sostenibile	Contenuti aggiuntivi senza modificare il core	Una nuova superficie data-only non richiede ricompilazione del gioco

Target primario: giocatori PC che apprezzano tattica a turni, hero-based games, deckbuilding leggero, mappe interattive e cooperazione. Il prodotto si rivolge sia a squadre coordinate sia a giocatori solitari che usano matchmaking e strumenti non vocali.

Target secondario: creator di mappe, designer di abilità, modder data-driven e community competitive interessate a regole personalizzate.

Esclusioni iniziali:

- nessun open world;
- nessun combattimento action in tempo reale;
- nessun equipaggiamento con rarità e statistiche crescenti;
- nessuna campagna narrativa completa nell'MVP;
- nessun supporto mobile al lancio;
- nessuna mod con codice nativo nelle partite pubbliche;
- nessun editor di mod distribuito prima della stabilizzazione della versione 1.0.

Configurazione competitiva proposta

Il target finale è **4 contro 4**, ma la produzione deve crescere gradualmente:

Fase	Configurazione	Motivazione
Prototipo	1 eroe contro bersagli o bot	Validazione di griglia, pathfinding e abilità
Vertical slice	1 contro 1 o 2 contro 2	Test del turno simultaneo con costi contenuti
MVP	2 contro 2 e 3 contro 3 privato	Validazione multiplayer e chiarezza della UI
Versione 1.0	4 contro 4	Esperienza completa e composizioni di squadra
Post-lancio	Regole custom	Dimensioni configurabili per lobby e mod

Core loop



Struttura proposta del turno

Fase	Durata indicativa	Attività
Osservazione	Integrata nella pianificazione	Esame di stato, minacce, LOS, terreno e cooldown
Pianificazione	25 secondi	Movimento, abilità, bersaglio, intento e ping
Lock	2-3 secondi	Validazione finale, risoluzione dei conflitti strutturali
Preparazione	Event-driven	Scudi, stance, trappole, effetti preventivi
Mobilità rapida	Event-driven	Blink, dash, salto, grapple
Movimento	Event-driven	Percorsi normali e attraversamento archi
Azione	Event-driven	Attacchi, cure, interazioni e abilità
Impatto	Event-driven	Proiettili, AoE, knockback e reazioni
Ambiente	Event-driven	Fuoco, elettricità, acqua, crolli, obiettivi
Cleanup	1-2 secondi	Durate, cooldown, vittoria, nuovo turno

Non è necessario imporre a tutte le abilità le stesse fasi. Ogni azione produce uno o più **eventi di risoluzione** con priorità, timestamp logico e dipendenze. Il server ordina questi eventi e ne distribuisce il risultato canonico.

Requisiti di gameplay e criteri di accettazione

Mappa multilivello

La mappa è un insieme di celle organizzate in livelli logici. Una coordinata è definita almeno da **X**, **Y** e **Level**, non semplicemente da una posizione nello spazio 3D. Due celle possono essere collegate anche quando non sono geometricamente adiacenti, per rappresentare scale, ascensori, teletrasporti, zipline, salti, portali o condotti.

Le celle contengono dati logici e riferimenti visuali, ma non devono necessariamente essere implementate come migliaia di Actor separati. La soluzione raccomandata è conservare i dati in array o strutture gestite da un **WorldSubsystem** o da un **GridManager**, usando mesh istanziate per la visualizzazione. In Unreal gli Actor possono essere composti da componenti, ma per un gran numero di celle è preferibile una composizione **data-oriented** invece di un **ActorComponent** UObject per ogni proprietà di ogni cella. Epic descrive gli Actor come oggetti replicabili e componibili, ma ciò non implica che ogni unità dati debba diventare un Actor. ⁴

Ogni cella deve poter descrivere:

- coordinate e livello;
- altezza tattica;
- tipo di superficie;
- costo base di attraversamento;
- tag semantici;
- copertura sui lati;
- blocco del movimento;
- blocco o attenuazione della vista;
- occupante;
- elementi interattivi;
- effetti persistenti;
- archi in entrata e uscita;
- revisione del grafo;
- versione serializzata.

Sistema di componenti logici delle celle

Il termine “componente” indica una struttura dati modulare, non obbligatoriamente un `UActorComponent`. Esempi:

```
Cell
├─ TransformFragment
├─ SurfaceFragment
├─ CoverFragment
├─ VisibilityFragment
├─ HazardFragment
├─ InteractionFragment
├─ OccupancyFragment
└─ DynamicEffectFragments[]
```

I tipi stabili e frequentemente interrogati possono essere campi diretti. Gli effetti opzionali o aggiunti da mod possono essere frammenti registrati mediante Gameplay Tags e definizioni dati. I Gameplay Tags di Unreal sono etichette gerarchiche progettate per identificare, classificare, filtrare e confrontare oggetti; sono quindi adatti a nomi come `Surface.Water`, `Hazard.Fire` o `Traversal.Climb`.

5

Pathfinding semantico

Il grafo tattico deve esporre un'API indipendente dalla rappresentazione visuale:

```
FindPath
    UnitTraversalProfile
    StartNode
    DestinationNode
    QueryContext
```

```
GraphRevision  
-> PathResult
```

Il costo di attraversamento è composto, non hard-coded:

```
Costo totale dell'arco =  
  costo base  
  + costo superficie  
  + differenza di altezza  
  + costo collegamento verticale  
  + effetto ambientale  
  + modificatore dell'eroe  
  + modificatore della build  
  + penalità di occupazione  
  + rischio opzionale
```

I provider iniziali sono:

Provider	Responsabilità
BaseTraversalProvider	Costo del terreno e blocchi assoluti
HeightProvider	Salita, discesa, caduta e requisiti
SurfaceProvider	Acqua, ghiaccio, fango, lava, vegetazione
HazardProvider	Danni o status previsti durante il passaggio
UnitAffinityProvider	Resistenze, immunità e preferenze dell'eroe
EquipmentProvider	Rampino, jetpack, stivali, scudo ambientale
OccupancyProvider	Celle occupate, prenotate o contese
TacticalProvider	Esposizione, copertura e influenza nemica, per bot
AbilityProvider	Regole temporanee prodotte da abilità
ModProvider	Estensioni data-driven con limiti e priorità dichiarate

Il pathfinder deve supportare due modalità:

- **Reachability:** quali celle sono raggiungibili con il budget disponibile;
- **Best path:** quale percorso valido minimizza il costo richiesto dalla query.

Per il giocatore, il costo deve privilegiare prevedibilità e controllo. Il pathfinder non deve scegliere autonomamente un percorso “più sicuro” se questo produce una traiettoria inaspettata senza mostrarla chiaramente. Per l'intelligenza artificiale può invece essere applicato un profilo tattico più ricco.

Unreal dispone di Navigation System, agenti differenti e link di navigazione, che possono assistere spostamenti e navigazione geometrica. Per RefactorTactics, tuttavia, la logica tattica deve restare in un grafo personalizzato perché il costo dipende da turno, build, pericoli futuri e regole semantiche non

puramente geometriche. Gli strumenti UE di navigazione possono essere usati sotto il livello tattico per allineare animazione e collisioni. 6

Line of sight separata

LOS, tiro e pathfinding devono essere sistemi distinti:

```
TraversalGraph -> dove posso andare  
VisibilityGraph -> cosa posso vedere  
ProjectileQuery -> cosa posso colpire e attraversare  
CoverQuery -> quale protezione si applica
```

Una cella può essere attraversabile ma bloccare la vista, visibile ma non attraversabile, oppure attenuare il tiro senza impedirlo. La LOS tattica deve usare una query discreta sulle celle attraversate e, quando necessario, un trace 3D per verificare geometria visuale e punti di aggancio.

La query deve restituire più di un booleano:

```
LineOfSightResult  
- Visible  
- Blocked  
- Partial  
- CoverValue  
- ObstructionCell  
- HeightDelta  
- TraversedEffects
```

Abilità che modificano il grafo

Le abilità non devono manipolare direttamente gli array interni del pathfinder. Producono comandi:

```
AddEdge  
RemoveEdge  
SetEdgeEnabled  
AddCelleEffect  
RemoveCelleEffect  
ChangeSurface  
ChangeHeightState  
SetVisionBlocker  
DestroyMapElement  
SpawnMapElement
```

Esempi:

Abilità	Mutazione
Muro di ghiaccio	Blocca archi e LOS su un confine

Abilità	Mutazione
Portale	Aggiunge un arco bidirezionale o monodirezionale
Rampino	Rende utilizzabili archi con tag <code>Traversal.Grapple</code>
Crollo	Disattiva celle e rimuove collegamenti
Congelamento	Converte <code>Surface.Water</code> in <code>Surface.Ice</code>
Campo gravitazionale	Aggiunge costo agli archi nella zona
Piattaforma mobile	Cambia periodicamente i collegamenti verticali
Fumo	Attenua LOS senza bloccare il movimento
Elettricità	Propaga un pericolo attraverso celle con tag conduttivi

Ogni modifica incrementa una `GraphRevision`. Le cache dei percorsi includono questa revisione, evitando di riutare risultati calcolati su uno stato obsoleto.

Personalizzazione degli eroi

Ogni eroe mantiene:

- silhouette e identità;
- statistiche base;
- tema meccanico;
- un'abilità distintiva non sostituibile;
- punti di forza e debolezze leggibili.

La build completa è composta da:

Slot	Scelta proposta	Funzione
Archetipo	Fisso	Identità primaria
Specializzazione	1 su 2–3	Orientamento del ruolo
Moduli abilità	1 per abilità modificabile	Cambiamenti funzionali
Talenti	3 su circa 10–12	Sinergie e compromessi
Equipaggiamento tattico	2	Gadget condivisi o specifici
Affinità mappa	1	Interazione con superfici o collegamenti
Ultimate	1 su 2, in una fase avanzata	Scelta ad alto impatto
Consumabile	0–1	Risorsa limitata del match

I moduli devono cambiare comportamento, non offrire un'opzione palesemente superiore. Esempio di modifiche a una granata:

- esplosione immediata con area ridotta;
- detonazione ritardata con area maggiore;

- sostituzione del danno con fumo;
- divisione in mine;
- applicazione di una superficie incendiaria;
- knockback senza danno diretto.

Il Gameplay Ability System di Unreal è pensato per abilità, attributi, effetti e relativo networking, ed è adatto come infrastruttura per eroi e status. L' `AbilitySystemComponent` gestisce abilità attivabili, attributi, eventi ed effetti; Gameplay Tags possono rappresentare requisiti, immunità e stati. ⁷

Affinità con la mappa

Le affinità sono opzioni di build, non bonus automatici legati alla rarità:

- ignora il costo extra delle rocce;
- attraversa l'acqua senza penalità;
- usa zipline o ascensori con costo ridotto;
- ottiene un beneficio controllato nelle celle infuocate;
- può attraversare una differenza di altezza aggiuntiva;
- riduce gli effetti delle celle elettrificate;
- vede attraverso una quantità limitata di fumo.

Ogni affinità deve avere almeno una mappa in cui è utile e almeno una condizione in cui non produce vantaggio.

Perk di squadra

La squadra sceglie un perk condiviso prima del match. Il perk non deve sostituire la composizione degli eroi.

Esempi:

- radar condiviso;
- sensori iniziali;
- kit medico;
- fortificazione iniziale;
- riduzione del costo della prima interazione;
- riserva energetica comune;
- una ricognizione iniziale di breve durata.

Nel matchmaking con giocatori casuali, la scelta può avvenire mediante voto; in caso di pareggio, si applica una regola deterministica visibile.

Pianificazione condivisa

Ogni piano contiene:

```
PlayerPlan
- TurnId
- Revision
- PlannedPath
```

- Destination
- AbilityId
- TargetData
- AreaPreview
- Facing
- IntentionTag
- ReadyState
- ClientTimestamp
- ServerValidationState

Gli alleati visualizzano:

- percorso previsto;
- ghost nella posizione finale;
- area d'effetto;
- bersaglio;
- direzione;
- ordine logico dell'azione;
- etichetta d'intenzione;
- stato "sta pensando", "sta modificando", "pronto";
- ultima modifica significativa;
- conflitti e avvisi.

Gli avversari visualizzano soltanto informazioni pubbliche definite dal design, come il fatto che un giocatore sia connesso o eventualmente che abbia confermato. Non ricevono coordinate, abilità, percorso, bersaglio o intenzione.

Disegno e comunicazione

Il giocatore può aggiungere annotazioni temporanee:

- frecce;
- cerchi;
- linee;
- ping contestuali;
- etichette "focus", "difendo", "scout", "trappola", "aspetto", "fuga".

Il disegno deve essere limitato per durata, frequenza e quantità per prevenire spam. Deve essere possibile mutare un giocatore senza disabilitare le informazioni strutturali essenziali, come percorso e AoE.

Common UI fornisce strumenti per UI stratificate, routing dell'input e supporto cross-platform; è un buon candidato per menu, layer di pianificazione, gamepad e schermate di build. 8

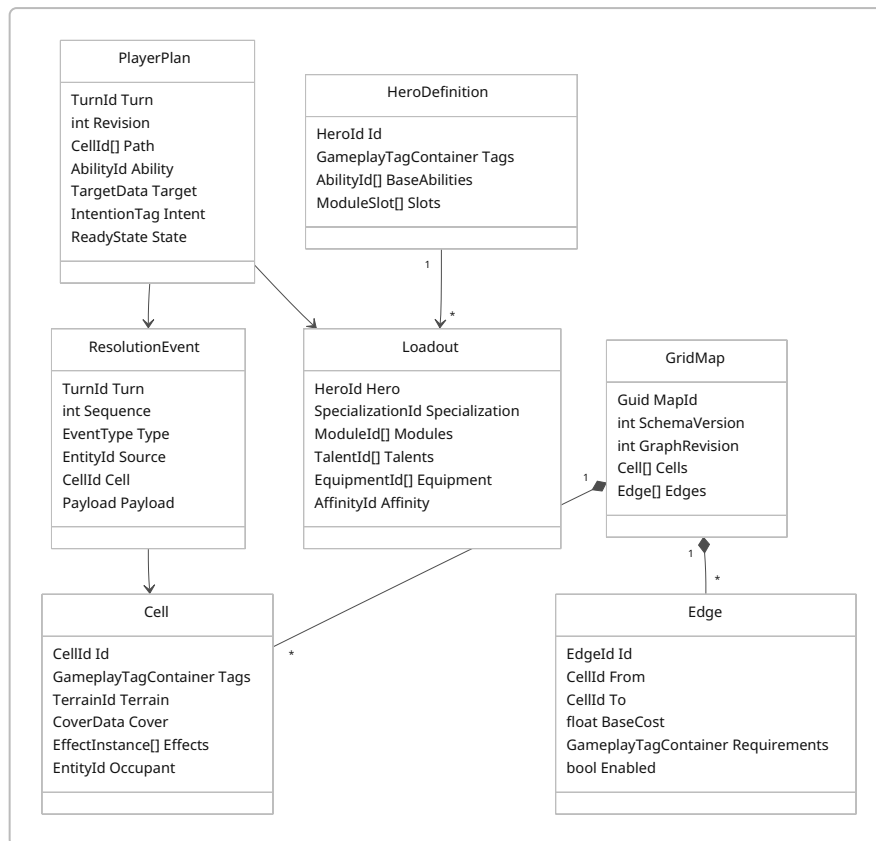
Criteri di accettazione delle feature

Feature	Criterio minimo verificabile	Fase
Mappa multilivello	Un eroe raggiunge un piano superiore tramite scala e un altro tramite abilità; i due collegamenti hanno costi e requisiti differenti	Vertical slice
Celle semantiche	Almeno cinque superfici modificano movimento o combattimento senza codice specifico nell'eroe	MVP
Cost provider	Aggiungere un provider non richiede modificare l'algoritmo A*	Vertical slice
Path dinamico	La distruzione di un ponte invalida il percorso entro 100 ms nella build di test	MVP
Reachability	L'anteprima mostra tutte e sole le celle raggiungibili con il budget corrente nei test golden	Vertical slice
LOS separata	Una cella attraversabile può bloccare la vista senza bloccare il path	Vertical slice
Cover	La UI indica la copertura applicata e il resolver usa lo stesso risultato	MVP
Mutazioni abilità	Muro, fumo e portali modificano sistemi diversi senza dipendenza circolare	MVP
Moduli eroe	Due build dello stesso eroe producono almeno due pattern di gioco distinti	MVP
Talenti	Il giocatore può selezionare tre talenti e il server ne valida compatibilità e budget	MVP
Equipaggiamento	Due gadget equipaggiabili sono salvati, replicati e utilizzabili	MVP
Affinità	Un'affinità modifica il costo tramite provider, non con controllo hard-coded nel pathfinder	MVP
Perk di squadra	La scelta è condivisa, validata e applicata a tutti i membri	Versione 1.0
Ghost alleati	Una modifica del piano compare agli alleati entro 150 ms più latenza di rete	MVP
Segretezza	Un client avversario e una cattura di rete non contengono il piano privato	MVP, bloccante
Avvisi	Collisione di destinazione e friendly fire potenziale producono warning non bloccanti	MVP
Disegno	Le annotazioni sono visibili solo alla squadra, rate-limited e cancellate a fine turno	Versione 1.0
Intenti	Almeno sei etichette sono selezionabili con mouse e gamepad	MVP
Stato pronto	Il turno può iniziare in anticipo quando tutti hanno confermato	MVP

Classe o sistema	Responsabilità RefactorTactics
<code>ARTGameMode</code>	Regole autoritative, join, team, inizio e fine match; esiste solo sul server
<code>ARTGameState</code>	Turno corrente, fase pubblica, timer sincronizzato, punteggi e stato pubblico
<code>ARTPlayerController</code>	Input locale, richieste RPC, selezione, camera e piano del giocatore
<code>ARTPlayerState</code>	Team, eroe pubblico, stato connessione e statistiche visibili
<code>ARTHeroPawn</code>	Rappresentazione fisica dell'eroe
<code>URGridWorldSubsystem</code>	Celle, coordinate, revisioni e query della mappa
<code>URTurnSubsystem</code>	Validazione dei piani e produzione degli eventi
<code>URContentRegistrySubsystem</code>	Definizioni di eroi, superfici, abilità, mod e versioni
<code>ATeamPlanReplicator</code>	Replica selettiva dei piani a una sola squadra
<code>URProfileSaveGame</code>	Preferenze, loadout, progressione e profili mod

Epic specifica che `GameMode` esiste solo sul server, mentre `GameState` contiene lo stato destinato a tutti i client. Per questo motivo il piano privato non deve essere memorizzato nel `GameState`. Analogamente, i `PlayerState` sono presenti e replicati su tutte le macchine, quindi non sono il posto corretto per coordinate e bersagli segreti. ¹⁰

Modello dati



Data Assets e registri

Definizioni stabili come eroi, abilità, terreni, moduli ed equipaggiamento devono essere `UPrimaryDataAsset` o record equivalenti registrati dall'Asset Manager. Un `PrimaryDataAsset` implementa un identificatore di asset primario e supporta bundle caricabili e scaricabili esplicitamente dall'Asset Manager. Le Data Registries possono integrare sorgenti provenienti da Game Feature Plugin, utile nella fase mod. ¹¹

Struttura C++ iniziale, copy-paste

File `Source/RefactorTactics/Public/Grid/RTGridTypes.h`:

```

#pragma once

#include "CoreMinimal.h"
#include "GameplayTagContainer.h"
#include "RTGridTypes.generated.h"

USTRUCT(BlueprintType)
struct REFACTORTACTICS_API FRTGridCoord
{
    GENERATED_BODY()

    UPROPERTY(EditAnywhere, BlueprintReadWrite)
    int32 X = 0;

```

```

UPROPERTY(EditAnywhere, BlueprintReadWrite)
int32 Y = 0;

UPROPERTY(EditAnywhere, BlueprintReadWrite)
int32 Level = 0;

bool operator==(const FRTGridCoord& Other) const
{
    return X == Other.X && Y == Other.Y && Level == Other.Level;
}
};

FORCEINLINE uint32 GetTypeHash(const FRTGridCoord& Coord)
{
    uint32 Hash = HashCombine(GetTypeHash(Coord.X), GetTypeHash(Coord.Y));
    return HashCombine(Hash, GetTypeHash(Coord.Level));
}

USTRUCT(BlueprintType)
struct REFACTORTACTICS_API FRTCellData
{
    GENERATED_BODY()

    UPROPERTY(EditAnywhere, BlueprintReadWrite)
    FRTGridCoord Coord;

    UPROPERTY(EditAnywhere, BlueprintReadWrite)
    FGameplayTagContainer Tags;

    UPROPERTY(EditAnywhere, BlueprintReadWrite, meta = (ClampMin = "0.0"))
    float BaseMovementCost = 1.0f;

    UPROPERTY(EditAnywhere, BlueprintReadWrite)
    bool bBlocksMovement = false;

    UPROPERTY(EditAnywhere, BlueprintReadWrite)
    bool bBlocksVision = false;
};

```

Questo snippet introduce soltanto il modello minimo. Gli effetti dinamici non devono essere aggiunti immediatamente alla struttura: saranno introdotti quando esiste già un test che dimostra la necessità.

Definizione data-driven di un terreno

File `Source/RefactorTactics/Public/Grid/RTTerrainDefinition.h`:

```

#pragma once

#include "CoreMinimal.h"
#include "Engine/DataAsset.h"

```

```

#include "GameplayTagContainer.h"
#include "RTTerrainDefinition.generated.h"

UCLASS(BlueprintType)
class REFACTORTACTICS_API URTTerrainDefinition : public UPrimaryDataAsset
{
    GENERATED_BODY()

public:
    UPROPERTY(EditAnywhere, BlueprintReadOnly, Category = "Terrain")
    FName TerrainId = NAME_None;

    UPROPERTY(EditAnywhere, BlueprintReadOnly, Category = "Terrain")
    FGameplayTagContainer Tags;

    UPROPERTY(EditAnywhere, BlueprintReadOnly, Category = "Movement",
        meta = (ClampMin = "0.0"))
    float MovementMultiplier = 1.0f;

    UPROPERTY(EditAnywhere, BlueprintReadOnly, Category = "Vision")
    bool bBlocksVision = false;

    UPROPERTY(EditAnywhere, BlueprintReadOnly, Category = "Combat")
    float CoverBonus = 0.0f;
};

```

Dopo la compilazione:

1. aprire il Content Browser;
2. scegliere **Add → Miscellaneous → Data Asset**;
3. selezionare `RTTerrainDefinition`;
4. creare `DA_Terrain_Water`;
5. impostare `TerrainId = Water`;
6. impostare `MovementMultiplier = 2.0`;
7. assegnare il tag `Surface.Water`.

I Data Asset sono pensati per conservare dati di sistema come asset del progetto; Epic consiglia Data Asset per oggetti che non devono essere istanziati e non richiedono ereditarietà complessa. ¹²

Bridge concettuale C#

Il seguente codice non è necessario nel runtime UE. Serve per apprendere e testare il dominio con concetti familiari:

```

public readonly record struct CellId(int X, int Y, int Level);

public sealed record CellData(
    CellId Id,
    float BaseCost,
    bool BlocksMovement,

```



```

        IReadOnlySet<string> Tags));

public sealed record UnitProfile(
    IReadOnlySet<string> Tags,
    float MovementBudget);

public interface ITraversalCostProvider
{
    float GetAdditionalCost(UnitProfile unit, CellData cell);
}

public sealed class WaterCostProvider : ITraversalCostProvider
{
    public float GetAdditionalCost(UnitProfile unit, CellData cell)
    {
        if (!cell.Tags.Contains("Surface.Water"))
            return 0f;

        return unit.Tags.Contains("Affinity.Aquatic") ? 0f : 1f;
    }
}

```

La traduzione mentale è:

C#	Unreal
<code>record struct</code>	<code>USTRUCT(BlueprintType)</code>
<code>class</code> dati configurabili	<code>UPrimaryDataAsset</code>
interfaccia	<code>UINTERFACE</code> e interface C++
attributo o string tag	<code>FGameplayTag</code>
evento C#	delegate UE o Blueprint Event
service singleton	<code>Subsystem</code>
prefab Unity	Blueprint Class, non equivalenza perfetta
GameObject component	Actor e ActorComponent

Separazione Blueprint/C++

Area	Blueprint	C++
Prototipi di abilità	Sì	Solo funzioni di supporto
UI e animazioni	Sì	Base e ottimizzazioni
Definizioni di contenuto	Data Asset e Blueprint	Tipi di base
Grafo e A*	Visualizzazione e debug	Sì
Resolver autoritativo	Configurazione	Sì

Area	Blueprint	C++
Replica team-only	Eventuali hook	Sì
Effetti GAS	Sì	Calcoli complessi
Mod registry	Pannelli editor	Sì
Tool esterni	Possibile Python	C# consigliato quando separato da UE

Epic consiglia di valutare Blueprint e C++ in base a prestazioni, accessibilità ai designer, data flow e manutenibilità, invece di scegliere un solo linguaggio per tutto. ¹³

Networking, sincronizzazione e modding

Modello di autorità

Il server è autorevole su:

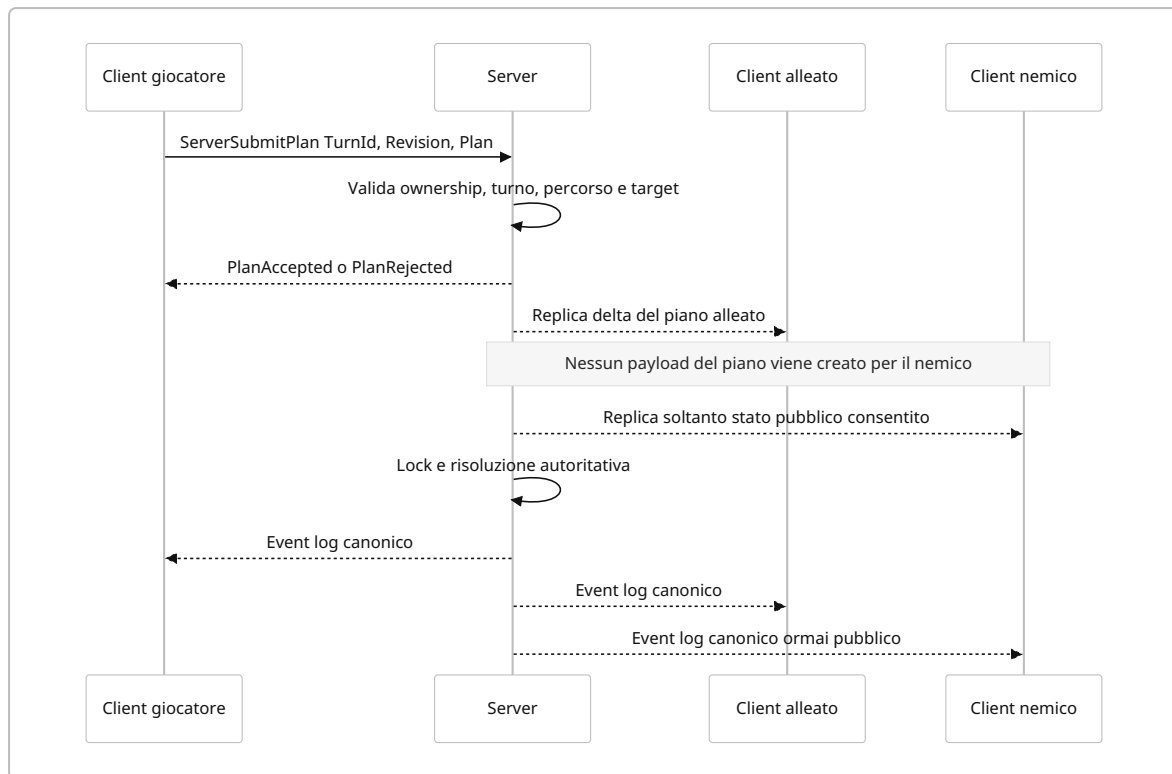
- composizioni di squadra;
- loadout validi;
- timer;
- stato della griglia;
- piani accettati;
- ordine degli eventi;
- danni, status ed eliminazioni;
- obiettivi e vittoria;
- revisioni del grafo;
- manifest delle mod.

Il client è autorevole soltanto su presentazione locale, input ancora non inviato, camera, accessibilità e anteprime dichiaratamente speculative.

La replica delle proprietà UE è server-to-client e affidabile nel senso che il client raggiungerà il valore più recente, ma non è garantito che osservi ogni valore intermedio. Per modifiche frequenti alla pianificazione, il sistema deve quindi usare revisioni, delta e semantica “ultimo piano valido”, non dipendere dall'osservazione di ogni singolo passaggio del cursore. ¹⁴

Replica privata del piano

Flusso raccomandato:



Implementazione proposta:

- `ARTPlayerController` invia una Server RPC;
- il server conserva il piano privato in una struttura non pubblica;
- un `ATeamPlanReplicator` per squadra espone i soli piani di quella squadra;
- `IsNetRelevantFor` o un Replication Graph personalizzato decide quali connessioni possono ricevere l'Actor;
- un Fast Array o struttura delta replica soltanto gli elementi modificati;
- il piano non compare in `GameState`, `PlayerState` o Actor globally relevant.

Unreal supporta Actor rilevanti solo per un owner e sistemi di relevancy; Fast Array Serializer è il wrapper previsto per array replicati con delta. Per una squadra intera, la normale ownership di un singolo PlayerController non basta, quindi è appropriata una relevancy personalizzata basata sul team.

15

Validazione di sicurezza obbligatoria

Il server verifica:

- che `TurnId` sia quello corrente;
- che la revisione sia più recente;
- che il giocatore controlli l'eroe;
- che il loadout sia valido;
- che il percorso sia contiguo nel grafo corrente;
- che ogni arco sia attraversabile;
- che il costo non ecceda il budget;
- che il bersaglio sia valido;
- che l'abilità non sia in cooldown;

- che le risorse siano disponibili;
- che il client non abbia inviato tag o costi arbitrari;
- che il piano non contenga riferimenti ad asset non registrati.

Sincronizzazione del turno

`GameState` replica:

```
CurrentTurnId
CurrentPhase
PhaseStartServerTime
PhaseEndServerTime
PublicReadyCount
GraphRevision
MatchState
```

Epic espone in `GameState` un tempo server sincronizzato tramite `GetServerWorldTimeSeconds`, adatto a calcolare un timer client senza affidarsi all'orologio locale. ¹⁶

Al lock:

1. il server crea un'istantanea dei piani accettati;
2. assegna una revisione immutabile;
3. genera gli eventi;
4. ordina gli eventi per fase, priorità e tie-break deterministico;
5. applica ogni evento allo stato autoritativo;
6. scrive un event log;
7. replica l'event log;
8. i client riproducono animazioni e VFX;
9. il server avanza alla fase successiva.

Il gioco non deve dipendere dalla determinazione identica della simulazione su ogni client. Il server produce il risultato canonico e i client presentano tale risultato. In caso di divergenza visuale, il client si riallinea allo snapshot successivo.

Riconnessione

Alla riconnessione, il server invia:

- manifest e hash dei contenuti;
- snapshot del match;
- turno e fase;
- stato delle celle dinamiche;
- unità e status;
- event log recente;
- eventuale piano privato del giocatore se la fase è ancora aperta.

Supporto alle mod

Il supporto viene diviso in livelli.

Livello	Contenuto	Contesto consentito
Data-only	Terreni, effetti parametrizzati, talenti, moduli, equipaggiamento	Offline, private e successivamente server approvati
Mappe	Celle, geometria, collegamenti, obiettivi	Offline e lobby con manifest coincidente
Blueprint sandbox	Abilità costruite su API esposte e validate	Offline e private, dopo audit
Regole di gioco	Varianti del turno, punteggio, roster	Server custom
Codice nativo	Plugin C++ arbitrari	Fuori scope per matchmaking pubblico

Unreal permette di separare asset cooked in pacchetti e `.pak`, e Game Features consente feature plugin attivabili o disattivabili. Tuttavia, l'inferenza progettuale è che RefactorTactics debba aggiungere manifest, dipendenze, hash, compatibilità, sandbox e regole di matchmaking: queste responsabilità non sono risolte automaticamente dal semplice packaging. ¹⁷

Manifest proposto

```
{
  "modId": "com.example.volcanic-pack",
  "displayName": "Volcanic Pack",
  "version": "1.2.0",
  "gameApiVersion": "1.0",
  "type": "DataOnly",
  "contentHash": "sha256-placeholder",
  "dependencies": [
    {
      "modId": "refactortactics.core",
      "version": ">=1.0.0"
    }
  ],
  "register": {
    "terrains": [
      "DA_Terrain_Lava"
    ],
    "cellEffects": [
      "DA_Effect_VolcanicGas"
    ]
  }
}
```

Regole di sicurezza delle mod

- nessun codice nativo scaricato automaticamente;
- caricamento soltanto all'avvio o prima dell'ingresso in lobby;

- niente hot-load durante un match competitivo;
- limiti su dimensione, numero di asset e complessità;
- whitelist delle classi Blueprint istanziabili;
- validazione server delle definizioni;
- ID namespace obbligatori;
- hash dell'intero profilo mod;
- salvataggi associati a `GameApiVersion` e versioni delle mod;
- impossibilità di entrare in matchmaking standard con mod attive;
- crash di una mod isolato, per quanto possibile, dalla corruzione dei profili;
- warning esplicito per contenuti non firmati.

Salvataggi

Unreal offre `USaveGame` come base per salvare stato in slot. Il profilo deve memorizzare identificatori stabili, non puntatori runtime: Hero ID, Module ID, Mod ID e versione. ¹⁸

```
ProfileSave
- SchemaVersion
- PlayerSettings
- AccessibilitySettings
- UnlockedHorizontalOptions
- SavedLoadouts
- PreferredIntentLabels
- ModProfiles
  - ModId
  - Version
  - Hash
  - Enabled
```

Le migrazioni sono funzioni esplicite:

```
Migrate v1 -> v2
Migrate v2 -> v3
```

Non è accettabile affidarsi ai nomi di cartelle degli asset come unico identificatore persistente.

Requisiti funzionali e non funzionali

Requisiti funzionali principali

ID	Requisito
FR-MATCH-01	Il server deve gestire una macchina a stati del match e del turno
FR-MATCH-02	Il turno deve terminare per scadenza o conferma di tutti i giocatori
FR-MATCH-03	Un piano non valido deve essere rifiutato con una motivazione utilizzabile dalla UI
FR-GRID-01	La mappa deve supportare almeno tre livelli logici

ID	Requisito
FR-GRID-02	Gli archi devono supportare direzione, costo, requisiti e stato abilitato
FR-GRID-03	Le mutazioni devono incrementare la revisione del grafo
FR-PATH-01	La reachability deve considerare unità, build ed effetti
FR-PATH-02	Il pathfinder deve essere indipendente dai cost provider
FR-LOS-01	LOS e movimento devono poter produrre risultati differenti
FR-ABILITY-01	Un'abilità deve poter modificare celle, archi o visibilità tramite comandi
FR-HERO-01	Il server deve validare slot, esclusioni e budget della build
FR-HERO-02	La progressione competitiva non deve modificare permanentemente HP o danni base
FR-PLAN-01	Gli alleati devono ricevere gli aggiornamenti del piano durante la pianificazione
FR-PLAN-02	Gli avversari non devono ricevere il payload del piano
FR-PLAN-03	La UI deve mostrare conflitti senza bloccare automaticamente decisioni valide
FR-MOD-01	Il gioco deve validare manifest, versione API, dipendenze e hash
FR-MOD-02	Il matchmaking standard deve rifiutare profili modificati
FR-SAVE-01	I loadout devono sopravvivere agli aggiornamenti tramite migrazioni versionate

Prestazioni

Target iniziali per una mappa di riferimento con circa 2.000–3.000 celle logiche:

Area	Target
Frame rate client	60 fps a 1080p sul PC target
Path preview singola	meno di 50 ms al percentile 95
Reachability completa	meno di 100 ms al percentile 95
Aggiornamento locale dopo mutazione	meno di 100 ms
Turn resolution server	meno di 250 ms di calcolo, esclusa presentazione
Aggiornamento piano alleato	meno di 150 ms più latenza effettiva
Snapshot riconnessione	meno di 5 secondi su connessione stabile
Memoria dati griglia	profilo misurato e privo di UObject per-frammento non necessari
Caricamento mappa	meno di 15 secondi su SSD target

Questi sono target di prodotto da misurare, non garanzie dell'engine.

Affidabilità

- nessuna azione può essere applicata due volte dopo riconnessione;
- ogni evento possiede ID univoco e sequenza;
- la risoluzione dello stesso snapshot con gli stessi piani deve produrre lo stesso event log sul server;
- salvataggi scritti in modo atomico o con backup;
- crash report con turno, revisione del grafo e manifest contenuti;
- replay tecnico sufficiente a riprodurre bug del resolver.

Accessibilità

- rimappatura completa di mouse, tastiera e gamepad;
- colori mai usati come unico segnale;
- pattern e icone differenti per percorsi alleati;
- spessore e opacità configurabili;
- modalità per daltonismo;
- riduzione di flash e camera shake;
- velocità della risoluzione configurabile localmente;
- sottotitoli e segnali testuali per eventi importanti;
- possibilità di disattivare disegni manuali conservando gli intenti strutturati.

Enhanced Input dispone di impostazioni per mapping utente e accessibilità; Common UI supporta routing dell'input e UI multiplatforma. ¹⁹

Osservabilità e debug

Il progetto deve includere overlay per:

- ID della cella;
- tag;
- costo per provider;
- archi attivi;
- revisione del grafo;
- LOS;
- cover;
- piani ricevuti;
- relevancy di rete;
- eventi del turno;
- hash dei contenuti;
- motivazione di invalidazione.

Esempio di tooltip debug:

```
Cell: 12, 8, Level 1
Base: 1.0
Water: +1.0
Affinity Aquatic: -1.0
Enemy Influence: +0.5
```


Test

Livello	Copertura
Unit test C++	Coordinate, cost provider, A*, LOS discreta, ordinamento eventi
Golden test	Reachability e LOS confrontate con risultati attesi
Functional test UE	Abilità, mappe, UI, obiettivi
Network test	Lag, perdita pacchetti, riconnessione, replica team-only
Security test	Packet inspection e tentativi di submit invalidi
Content validation	ID duplicati, tag mancanti, dipendenze cicliche
Soak test	Match bot ripetuti su dedicated server
Mod compatibility	Manifest errati, versioni incompatibili, asset mancanti
Accessibility QA	Contrasto, gamepad, scaling e segnali non cromatici

Unreal dispone di un framework di automazione e test eseguibili dall'editor; questi test vanno integrati nella CI insieme a build cooked e dedicated server. ²⁰

Definition of Done generale

Una feature non è completata quando “funziona in editor”. È completata quando:

- ha criterio di accettazione;
- funziona in standalone e multiplayer;
- è validata dal server;
- ha almeno un test automatico;
- possiede debug visualization;
- non trasmette dati segreti;
- usa ID e serializzazione versionata;
- è provata in una build packaged;
- è documentata;
- non richiede asset temporanei non licenziati.

Percorso didattico e setup

Il percorso didattico deve produrre in modo incrementale il gioco. Ogni lezione introduce pochi concetti e termina con un risultato visibile.

Link ufficiali essenziali

- [Unreal Engine 5.8 Release Notes](#) ²¹
- [Setup di Visual Studio per Unreal Engine](#) ²²
- [Gameplay Framework](#) ²³

- [Game Features e Modular Gameplay](#) ²⁴
- [Gameplay Ability System](#) ²⁵
- [Gameplay Tags](#) ²⁶
- [Lyra Sample Game](#) ²⁷
- [Common UI](#) ²⁸
- [Accesso al sorgente Unreal su GitHub](#) ²⁹
- [Guida Unreal per sviluppatori Unity](#) — è basata su UE4, ma rimane utile per la traduzione concettuale da un ecosistema C# a Unreal. ³⁰

Configurazione Windows consigliata

1. Installare Epic Games Launcher.
2. Installare Unreal Engine 5.8.
3. Installare Visual Studio 2022.
4. Nel Visual Studio Installer selezionare:
5. **Game development with C++;**
6. tool C++;
7. Windows SDK;
8. strumenti Unreal disponibili;
9. AddressSanitizer opzionale.
10. Installare Git.
11. Installare Git LFS.
12. Creare un repository privato.
13. Aggiungere un `.gitignore` Unreal.
14. Tracciare con Git LFS almeno `.uasset`, `.umap`, file audio e asset binari pesanti.
15. Creare un progetto **Games** → **Blank** → **C++** → **Desktop** chiamato esattamente `RefactorTactics`.
16. Disabilitare Starter Content, salvo necessità didattiche.
17. Aprire il progetto una volta e verificare la compilazione.
18. Attivare inizialmente:
 - Enhanced Input;
 - Gameplay Tags;
 - Gameplay Abilities;
 - Gameplay Tasks;
 - Common UI.
19. Attivare Game Features e Modular Gameplay solo nella lezione dedicata, evitando di introdurre troppa complessità nella prima settimana.
20. Creare una build packaged vuota prima di scrivere sistemi importanti.

La guida Epic per Visual Studio richiede il workload di sviluppo giochi C++ e i relativi strumenti; l'installazione dell'engine tramite Launcher include i prerequisiti runtime comuni. ³¹

Struttura cartelle iniziale

```
Content/
├── Core/
│   ├── Blueprints/
│   └── Input/
```

```

|   |─ UI/
|   |─ Data/
|─ Grid/
|   |─ Blueprints/
|   |─ Materials/
|   |─ Meshes/
|   |─ Data/
|─ Heroes/
|─ Abilities/
|─ Maps/
|─ Tests/
|─ Developer/

Source/RefactorTactics/
|─ Core/
|─ Grid/
|─ Pathfinding/
|─ Combat/
|─ Turns/
|─ Networking/
|─ Content/
|─ Save/

```

Sequenza delle lezioni

Lezione	Risultato giocabile	Concetti
Ambiente e primo progetto	Build vuota avviabile	Launcher, Visual Studio, Editor, Git
Mentalità Unreal per C# developer	Actor controllabile	UObject, Actor, Component, Pawn, Controller
Camera tattica	Pan, zoom e rotazione	Enhanced Input, Pawn camera
Prima cella	Cella selezionabile	Blueprint, collision, mouse trace
Griglia generata	Mappa 10×10	USTRUCT, array, coordinate
Movimento semplice	Eroe spostato su celle	Selezione, budget, preview
A* in C#	Pathfinder verificato fuori UE	Algoritmo e test
A* in C++	Percorso visualizzato in UE	Priority queue, graph API
Mappa multilivello	Scale tra due piani	Edge, requisiti, layer
Terreni data-driven	Acqua e fango cambiano il costo	Data Asset, Gameplay Tags
Cost provider	Eroi differenti scelgono costi differenti	Interface e registry
LOS e cover	Anteprima di tiro	Trace, altezza, cover

Lezione	Risultato giocabile	Concetti
Primo eroe	Abilità base funzionante	GAS, attributi, effetti
Mappa modificabile	Muro e fumo	Graph commands, revision
Turno simultaneo locale	Due unità pianificano e risolvono	State machine, event log
UI di pianificazione	Ghost, path, AoE e ready	UMG/Common UI
Multiplayer base	Due client con stato condiviso	GameMode, GameState, RPC
Piani team-only	Alleato vede, nemico no	Relevancy e replica
Warning e intenti	Collisioni e focus marker	Validatori non bloccanti
Loadout eroe	Moduli, talenti e gadget	Primary Data Assets
Perk e affinità	Modificatori team e mappa	Provider componibili
Bot tattico	Bot completa un turno	Influence map, query
Packaging e dedicated server	Match packaged	Cook, server, logging
Architettura mod	Mod data-only di esempio	Manifest, registry, hash
Mod Kit finale	Nuova superficie e mappa esterna	Game Feature, pacchetto e validazione

Esempio step-by-step: prima cella Blueprint

1. Creare un Blueprint Actor chiamato `BP_CellPreview`.
2. Aggiungere un `StaticMeshComponent`.
3. Assegnare un cubo piatto.
4. Abilitare collisione `Query Only`.
5. Creare le variabili `X`, `Y`, `Level`.
6. Marcarle `Instance Editable`.
7. Nel PlayerController abilitare mouse cursor e click events.
8. Su input click:
9. chiamare `Get Hit Result Under Cursor`;
10. ottenere `Hit Actor`;
11. eseguire cast a `BP_CellPreview`;
12. stampare `X`, `Y`, `Level`.
13. Creare un materiale con parametro scalare `Selected`.
14. Al passaggio del cursore impostare `Selected = 1`.
15. All'uscita impostare `Selected = 0`.

Risultato: il giocatore può puntare una cella e leggere le sue coordinate senza C++.

Esempio step-by-step: cost provider in C#

File console temporaneo:

```
var cell = new CellData(  
    new CellId(2, 3, 0),  
    BaseCost: 1f,  
    BlocksMovement: false,  
    Tags: new HashSet<string> { "Surface.Water" });  
  
var normalUnit = new UnitProfile(  
    new HashSet<string>(),  
    MovementBudget: 5f);  
  
var aquaticUnit = new UnitProfile(  
    new HashSet<string> { "Affinity.Aquatic" },  
    MovementBudget: 5f);  
  
var provider = new WaterCostProvider();  
  
Console.WriteLine(cell.BaseCost +  
    provider.GetAdditionalCost(normalUnit, cell));  
// 2  
  
Console.WriteLine(cell.BaseCost +  
    provider.GetAdditionalCost(aquaticUnit, cell));  
// 1
```

Obiettivo didattico: comprendere che il terreno non deve chiedere “sei il personaggio acquatico?”, e il pathfinder non deve conoscere l’acqua. Il provider traduce la relazione tra tag.

Esempio step-by-step: primo turno locale

1. Creare enum Blueprint `ERTTurnPhase` :
2. Planning;
3. Locked;
4. Resolving;
5. Cleanup.
6. Creare `BP_TurnManager` .
7. Aggiungere variabile `CurrentPhase` .
8. Aggiungere `TurnId` .
9. Aggiungere array `SubmittedPlans` .
10. Creare evento `SubmitPlan` .
11. Durante `Planning` , sostituire il piano precedente della stessa unità.
12. Quando tutte le unità sono pronte:
13. impostare `Locked` ;

14. copiare `SubmittedPlans` in `LockedPlans` ;
15. ordinare per fase logica;
16. impostare `Resolving` ;
17. eseguire ogni piano;
18. impostare `Cleanup` ;
19. incrementare `TurnId` ;
20. tornare a `Planning` .
21. Testare con due unità nello stesso processo prima di aggiungere networking.

C# binding opzionale

UnrealSharp può essere studiato in un branch separato dopo il primo vertical slice:

experiment/unrealsharp

Criteri prima di adottarlo nel progetto principale:

- build editor e packaged ripetibile;
- compatibilità con la versione UE fissata;
- supporto dedicated server;
- supporto delle piattaforme target;
- interoperabilità con Gameplay Tags e GAS;
- stack trace e debugging sufficienti;
- CI funzionante;
- licenza compatibile;
- piano di uscita che consenta di migrare il codice critico.

Fino a quando questi criteri non sono verificati, il core rimane C++/Blueprint.

Roadmap, risorse e rischi

Priorità

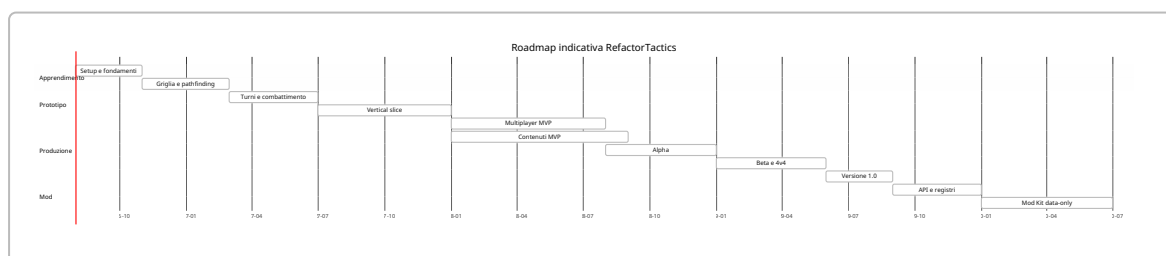
Priorità	Significato
P0	Necessario per dimostrare il gioco
P1	Necessario per una versione commerciale completa
P2	Importante ma rinviabile
P3	Post-lancio o sperimentale

Roadmap di prodotto

Milestone	Durata media indicativa	Deliverable	Feature dominanti
Fondamenta e apprendimento	2-3 mesi	Sandbox UE giocabile	Camera, cella, griglia, Git, build

Milestone	Durata media indicativa	Deliverable	Feature dominanti
Prototipo tattico	3-4 mesi	1v1 locale greybox	Pathfinding, LOS, abilità, turno
Vertical slice	4-6 mesi	Match breve 2v2 locale/rete LAN	Due livelli, tre superfici, tre eroi
Multiplayer MVP	5-8 mesi	2v2/3v3 private online	Server authority, team plans, reconnect
Content MVP	4-6 mesi	Roster e mappe testabili	4-6 eroi, due mappe, loadout
Alpha	4-6 mesi	Test esterni chiusi	UX, bilanciamento, bot, telemetry
Beta	3-5 mesi	Feature complete	4v4, matchmaking, accessibilità
Versione 1.0	2-4 mesi	Release PC	Polish, sicurezza, performance
Fondamenta mod	3-5 mesi	API e manifest interni	Registri, versionamento, validatori
Mod Kit	4-8 mesi	Tool pubblici data-only	Terreni, abilità parametrizzate, mappe
Mod estese	Variabile	Blueprint sandbox/private	Regole custom e contenuti avanzati

Le milestone si sovrappongono per team medi o grandi, ma non per un singolo sviluppatore.



Scope per release

Feature	Prototipo	MVP	Versione 1.0	Post-lancio
Griglia multilivello	Base	Sì	Rifinita	Estendibile
Cost provider	Base	Sì	Profilati	API mod
LOS e cover	Base	Sì	Rifinite	Custom
Mutazioni di mappa	Due esempi	Sì	Ampio set	Estendibili

Feature	Prototipo	MVP	Versione 1.0	Post-lancio
Turni simultanei	Locale	Online	Reconnect/replay	Regole custom
Piani alleati	Locale	Team-only	Disegno completo	API
Eroi	2-3	4-6	8-12	Roster esteso
Moduli e talenti	Parziale	Sì	Completi	Moddabili
Equipaggiamento	Minimo	Sì	Completo	Moddabile
Affinità	Una	Più opzioni	Bilanciate	Moddabili
Perk squadra	No	Base	Completo	Moddabile
Matchmaking	No	Private/sessioni	Pubblico	Server custom
Mod	No	Solo architettura interna	Nessuna UI pubblica	Mod Kit

Scenari di team e tempi

Le stime assumono PC come prima piattaforma, uso di asset temporanei o acquistati, nessuna campagna narrativa completa e nessun supporto mod pubblico prima della 1.0.

Scenario	Composizione	Vertical slice	MVP online	Versione 1.0
Piccolo	2-4 persone	10-15 mesi	22-32 mesi	36-54 mesi
Medio	7-12 persone	6-9 mesi	13-20 mesi	22-32 mesi
Grande	18-30 persone	4-7 mesi	9-15 mesi	16-24 mesi
Solo developer	1 persona più contractor	14-24 mesi	36-60 mesi	Oltre 60 mesi o scope ridotto

Per un progetto principalmente didattico e personale, la destinazione realistica iniziale è il **vertical slice**, non la versione 1.0 completa. Il vertical slice deve già mostrare la tesi del gioco: mappa multilivello, mutazione ambientale, build differenti e piano condiviso.

Ruoli chiave

Ruolo	Piccolo	Medio	Responsabilità
Product/game designer	1 condiviso	1-2	Regole, eroi, mappe, bilanciamento
Gameplay programmer	1	2-4	Turni, abilità, pathfinding

Ruolo	Piccolo	Medio	Responsabilità
Network/backend programmer	Contractor o condiviso	1-2	Autorità, sessioni, reconnect, sicurezza
UI/UX designer	Part-time	1-2	Planning UI, loadout, accessibilità
Technical artist	Contractor	1	Shader, VFX, pipeline e performance
Environment/level designer	1 condiviso	1-3	Mappe, celle, leggibilità
Character artist/animator	Contractor	1-3	Eroi, animazioni, silhouette
QA/automation	Sviluppatori	1-3	Test, regressioni e network matrix
Producer	Fondatore	1	Scope, milestone, dipendenze
Community/mod tools	Non prima della 1.0	1-2 post-lancio	SDK, documentazione e moderazione

Rischi principali

Rischio	Probabilità	Impatto	Mitigazione
Scope eccessivo	Alta	Critico	Vertical slice con massimo tre eroi, una mappa e poche superfici
UI incomprensibile	Alta	Critico	Prototipare ghost e warning prima dell'art finale
Troppe interazioni ambientali	Alta	Alto	Matrice di interazione esplicita e limite alle combinazioni MVP
Pathfinding lento	Media	Alto	Cache per revisione, query asincrone, profiling e provider semplici
Informazioni segrete esposte	Media	Critico	Team relevancy, packet inspection e test bloccante
Risoluzione ambigua	Alta	Alto	Event log, priorità pubbliche e tooltip "perché"
GAS troppo complesso per l'apprendimento	Media	Medio	Abilità Blueprint semplici prima di introdurre GAS completo
Dipendenza prematura da C# middleware	Media	Alto	Core ufficiale C++/Blueprint, branch sperimentale separato
Bilanciamento delle build	Alta	Alto	Budget, trade-off, telemetria e niente power progression
Mod che rompono salvataggi	Alta	Alto	Manifest, versioni, hash e migrazioni

Rischio	Probabilità	Impatto	Mitigazione
Mod malevole	Media	Critico	Data-only prima, no native code, matchmaking separato
Asset binari in conflitto	Alta	Medio	Ownership delle mappe, cartelle developer, Git LFS e mappe modulari
Upgrade UE problematico	Media	Alto	Versione fissata e upgrade solo su branch dedicato
Costi contenuto eroi	Alta	Alto	Stile visivo sostenibile e roster limitato
Multiplayer introdotto tardi	Media	Critico	Primo test a due client entro il vertical slice

Gate decisionali

Il progetto non deve avanzare alla milestone successiva finché il gate precedente non è superato.

Gate	Condizione
Gate griglia	Le coordinate, la selezione e il path sono affidabili
Gate tesi di gioco	Un playtester comprende perché la mappa cambia la decisione
Gate turno	Due unità risolvono piani simultanei senza stato incoerente
Gate rete	Due client completano dieci turni senza divergenza
Gate privacy	Il client nemico non riceve alcun piano privato
Gate MVP	Una partita completa è giocabile dall'ingresso al risultato
Gate 1.0	Performance, accessibilità, reconnect e matchmaking superano QA
Gate mod	Tutti i contenuti core sono già caricati attraverso API registrate
Gate Mod Kit	Una mod incompatibile non può corrompere match o profilo

Decisioni consolidate

Tema	Decisione
Engine	Unreal Engine 5.8 fissato per milestone
Linguaggi	Blueprint più core C++; C# per prototipi e tooling
C# runtime	Non nel percorso critico MVP
Autorità	Server authoritative
Match finale	4v4
MVP	2v2/3v3 privato
Mappa	Grafo logico multilivello indipendente dalla sola geometria UE

Tema	Decisione
Pathfinding	A* o variante su grafo con cost provider
LOS	Sistema separato
Celle	Dati compatti; non un Actor per ogni frammento
Abilità	GAS e comandi di mutazione della mappa
Customizzazione	Orizzontale, senza power progression competitiva
Pianificazione	Visibile live agli alleati
Sicurezza piani	Dati mai inviati ai client nemici
Modding	Ultima feature pubblica
Prima forma di mod	Data-only e mappe
Mod con codice nativo	Escluse dal matchmaking pubblico
Obiettivo didattico	Ogni lezione produce una build verificabile
Primo traguardo realistico	Vertical slice che dimostra la tesi del prodotto

1 24 Game Features and Modular Gameplay in Unreal Engine

https://dev.epicgames.com/documentation/unreal-engine/game-features-and-modular-gameplay-in-unreal-engine?utm_source=chatgpt.com

2 13 Balancing Blueprint and C++ | Unreal Engine 4.27 ...

https://dev.epicgames.com/documentation/en-us/unreal-engine/balancing-blueprint-and-cplusplus?application_version=4.27&utm_source=chatgpt.com

3 UnrealSharp: Home

https://www.unrealsharp.com/?utm_source=chatgpt.com

4 Unreal Python 4.27 (Experimental) documentation

https://dev.epicgames.com/documentation/en-us/unreal-engine/python-api/class/Actor?application_version=4.27&highlight=is_hidden_ed&utm_source=chatgpt.com

5 26 Gameplay Tags | Unreal Engine 4.27 Documentation

https://dev.epicgames.com/documentation/en-us/unreal-engine/gameplay-tags?application_version=4.27&utm_source=chatgpt.com

6 unreal.NavigationSystemV1

https://dev.epicgames.com/documentation/en-us/unreal-engine/python-api/class/NavigationSystemV1?application_version=4.27&utm_source=chatgpt.com

7 25 Gameplay Ability System | Unreal Engine 4.27 ...

https://dev.epicgames.com/documentation/en-us/unreal-engine/gameplay-ability-system?application_version=4.27&utm_source=chatgpt.com

8 28 Common UI Plugin for Advanced User Interfaces in Unreal ...

https://dev.epicgames.com/documentation/unreal-engine/common-ui-plugin-for-advanced-user-interfaces-in-unreal-engine?utm_source=chatgpt.com

9 21 Unreal Engine 5.8 Release Notes

https://dev.epicgames.com/documentation/unreal-engine/unreal-engine-5-8-release-notes?utm_source=chatgpt.com

10 **Gameplay Framework Quick Reference in Unreal Engine**

https://dev.epicgames.com/documentation/unreal-engine/gameplay-framework-quick-reference-in-unreal-engine?utm_source=chatgpt.com

11 **unreal.PrimaryDataAsset**

https://dev.epicgames.com/documentation/en-us/unreal-engine/python-api/class/PrimaryDataAsset?application_version=5.4&utm_source=chatgpt.com

12 **Unreal Python 5.5 (Experimental) documentation**

https://dev.epicgames.com/documentation/en-us/unreal-engine/python-api/class/DataAsset?application_version=5.5&utm_source=chatgpt.com

14 **Property Replication in Unreal Engine**

https://dev.epicgames.com/documentation/en-us/unreal-engine/property-replication-in-unreal-engine?application_version=5.2&utm_source=chatgpt.com

15 **Unreal Python 5.6 (Experimental) documentation**

https://dev.epicgames.com/documentation/en-us/unreal-engine/python-api/class/FastArraySerializer?application_version=5.6&utm_source=chatgpt.com

16 **Game Mode and Game State in Unreal Engine**

https://dev.epicgames.com/documentation/unreal-engine/game-mode-and-game-state-in-unreal-engine?utm_source=chatgpt.com

17 **Assets and Packages | Unreal Engine 4.27 Documentation**

https://dev.epicgames.com/documentation/en-us/unreal-engine/assets-and-packages?application_version=4.27&utm_source=chatgpt.com

18 **Unreal Python 5.6 (Experimental) documentation**

https://dev.epicgames.com/documentation/en-us/unreal-engine/python-api/class/SaveGame?application_version=5.6&utm_source=chatgpt.com

19 **unreal.EnhancedInputUserSettings**

https://dev.epicgames.com/documentation/en-us/unreal-engine/python-api/class/EnhancedInputUserSettings?application_version=5.4&utm_source=chatgpt.com

20 **Unreal Python 5.2 (Experimental) documentation**

https://dev.epicgames.com/documentation/en-us/unreal-engine/python-api/class/InterchangeImportTestPlan?application_version=5.2&utm_source=chatgpt.com

22 31 **Setting Up Visual Studio for Unreal Engine**

https://dev.epicgames.com/documentation/en-us/unreal-engine/setting-up-visual-studio-for-unreal-engine?application_version=4.27&utm_source=chatgpt.com

23 **Gameplay Framework in Unreal engine**

https://dev.epicgames.com/documentation/unreal-engine/gameplay-framework-in-unreal-engine?utm_source=chatgpt.com

27 **Lyra Sample Game in Unreal Engine**

https://dev.epicgames.com/documentation/unreal-engine/lyra-sample-game-in-unreal-engine?utm_source=chatgpt.com

29 **Unreal Engine on GitHub**

https://www.unrealengine.com/ue-on-github?utm_source=chatgpt.com

30 **Unreal Engine 4 For Unity Developers**

https://dev.epicgames.com/documentation/en-us/unreal-engine/unreal-engine-4-for-unity-developers?application_version=4.27&utm_source=chatgpt.com